# Laporan Tugas Besar Milestone 1

IF2224 Teori Bahasa Formal dan Otomata

Lexical Analysis

Semester II Tahun 2025/2026

**Disusun oleh:**

13524041 - Nathan Adhika Santosa

13524053 - Muhammad Haris Putra Sulastianto

13524085 - Ariel Cornelius Sitorus

13524091 - Vara Azzara Ramli Pulukadang

Laboratorium Ilmu Rekayasa dan Komputasi

Sekolah Teknik Elektro dan Informatika

Program Studi Teknik Informatika

Institut Teknologi Bandung

# Daftar Isi

**1 Teori Singkat** **3**
1.1 Lexical Analysis . . . 3
1.2 Token . . . 3
1.3 Deterministic Finite Automata (DFA) . . . 4
1.3.1 Karakteristik DFA . . . 5
1.3.2 Dead State . . . 5
1.4 Peran DFA dalam Lexical Analysis . . . 5

**2 Perancangan dan Implementasi** **7**
2.1 Perancangan DFA . . . 7
2.2 Penjelasan Diagram DFA . . . 10
2.3 Justifikasi Desain DFA . . . 10
2.4 Perancangan dan Implementasi Program . . . 12
2.4.1 Modul DFA . . . 12
2.4.2 Modul Lexer . . . 13
2.4.3 Modul Token . . . 14
2.4.4 Modul Utils . . . 19
2.4.5 Main Program . . . 19
2.4.6 Alur Eksekusi Program . . . 21

**3 Pengujian** **21**
3.1 Uji 1 . . . 21
3.2 Uji 2 . . . 22
3.3 Uji 3 . . . 23
3.4 Uji 4 . . . 24
3.5 Uji 5 . . . 26

**4 Kesimpulan dan Saran** **28**
4.1 Kesimpulan . . . 28
4.2 Saran . . . 28

**A Lampiran** **29**
A.1 Pranala Ke Repositori . . . 29
A.2 Workspace Diagram Transisi DFA . . . 29

A.3 Pembagian Tugas . . . . . 29

**Referensi** **30**

# 1 Teori Singkat

## 1.1 Lexical Analysis

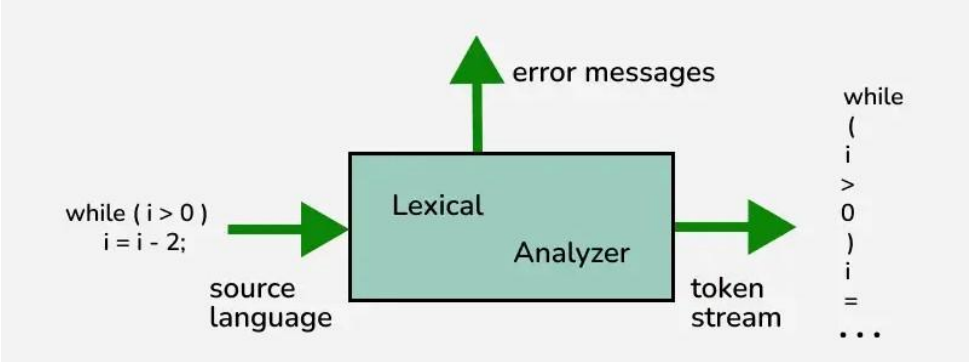


Figure 1: Lexical Analysis [3]

Lexical Analysis adalah tahap pertama dalam proses kompilasi atau interpretasi bahasa pemrograman. Pada tahap ini, program membaca *source code* yang berupa rangkaian karakter dan mengubahnya menjadi unit-unit bermakna yang disebut token.

Tujuan utama dari lexical analysis adalah menyederhanakan proses analisis selanjutnya (*syntax analysis*) dengan mengelompokkan karakter menjadi token yang terstruktur. Dengan demikian, tahap berikutnya tidak perlu lagi memproses karakter mentah secara langsung.

Lexer bekerja dengan membaca input secara berurutan dari kiri ke kanan, satu karakter pada satu waktu, dan mencocokkannya dengan pola tertentu untuk membentuk token.

Selain itu, lexical analysis juga bertanggung jawab untuk mengabaikan spasi, tab, dan *newline* yang tidak bermakna, menghapus komentar dari *source code*, serta mendeteksi kesalahan awal seperti simbol yang tidak dikenali.

Proses ini umumnya diimplementasikan menggunakan Deterministic Finite Automata (DFA), di mana setiap karakter input menyebabkan perpindahan state hingga suatu token dikenali.

## 1.2 Token

Token adalah unit terkecil yang memiliki makna dalam sebuah program hasil dari proses lexical analysis. Setiap token merepresentasikan kategori tertentu dari rangkaian karakter dalam source code.

Token umumnya direpresentasikan dalam bentuk pasangan *(type, value)*, di mana:

- *type* menyatakan jenis token
- *value* menyatakan nilai atau isi token

Contoh representasi token:

- `ident (a)`
- `intcon (10)`
- `plus (+)`

Dalam bahasa pemrograman Arion, token dapat diklasifikasikan menjadi beberapa kategori utama:

- **Keyword**: `program`, `var`, `begin`, `end`, dll
- **Identifier**: nama variabel, fungsi, atau konstanta
- **Operator**: `+`, `-`, `*`, `/`, `:=`, dll
- **Literal**: `intcon`, `realcon`, `string`, `charcon`
- **Delimiter**: `;`, `,`, `(`, `)`, dll

## 1.3 Deterministic Finite Automata (DFA)

Deterministic Finite Automata (DFA) adalah model komputasi yang digunakan untuk mengenali bahasa formal. DFA bersifat deterministik, artinya untuk setiap pasangan state dan simbol input, hanya terdapat satu kemungkinan transisi.

Secara formal, DFA didefinisikan sebagai 5-tuple $(Q, \Sigma, \delta, q_0, F)$, dengan:

- $Q$ adalah himpunan state
- $\Sigma$ adalah himpunan simbol input (alfabet)
- $\delta$ adalah fungsi transisi, $\delta : Q \times \Sigma \to Q$
- $q_0 \in Q$ adalah state awal
- $F \subseteq Q$ adalah himpunan state akhir (*accepting states*)

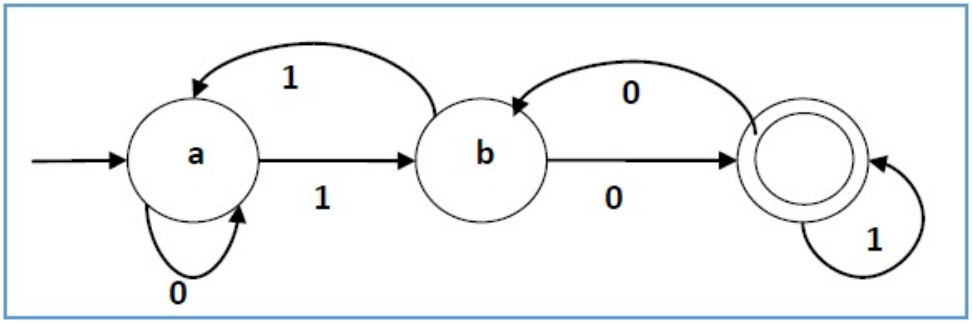


Figure 2: Deterministic Finite Automata [5]

DFA dapat direpresentasikan dalam bentuk graf berarah (*directed graph*) yang disebut *state diagram*. Dalam representasi ini:

- Setiap simpul (*vertex*) merepresentasikan sebuah state
- Setiap sisi berarah (*arc*) menunjukkan transisi antar state
- Label pada sisi menyatakan simbol input yang menyebabkan transisi
- State awal ditandai dengan panah masuk tanpa asal
- State akhir ditandai dengan lingkaran ganda

### 1.3.1 Karakteristik DFA

DFA memiliki beberapa karakteristik utama sebagai berikut:

- Bersifat deterministik, yaitu untuk setiap pasangan state dan simbol input hanya terdapat satu transisi yang mungkin
- Tidak memiliki $\epsilon$-transition (transisi tanpa input)
- Selalu berada pada satu state aktif pada setiap waktu
- Memproses input secara sekuensial dari kiri ke kanan

Karakteristik ini membuat DFA efisien dan cocok digunakan dalam proses lexical analysis.

### 1.3.2 Dead State

Dead state adalah state dalam DFA yang tidak dapat mencapai state akhir (*accepting state*). Jika DFA mencapai dead state, maka input yang sedang diproses tidak dapat membentuk token yang valid.

Biasanya, dead state digunakan untuk menangani kondisi input yang tidak sesuai dengan pola token yang telah didefinisikan.

## 1.4 Peran DFA dalam Lexical Analysis

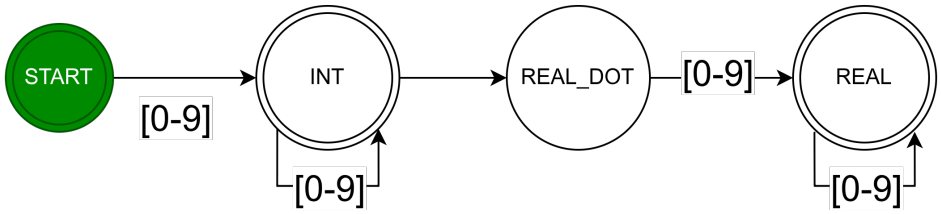


Figure 3: Contoh DFA untuk mengenali angka

Dalam lexical analysis, DFA digunakan sebagai mekanisme utama untuk mengenali pola token dari rangkaian karakter input. Proses ini dilakukan dengan membaca input secara

berurutan dari kiri ke kanan dan menentukan perpindahan state berdasarkan simbol yang dibaca.

Setiap jenis token direpresentasikan sebagai jalur tertentu dalam DFA. Sebuah token dianggap dikenali ketika proses mencapai state akhir (*accepting state*). Setelah itu, lexer akan menghasilkan token yang sesuai dan kembali ke state awal untuk melanjutkan pembacaan input berikutnya.

Karena sifatnya deterministik, DFA menjamin bahwa untuk setiap simbol input pada suatu state, hanya ada satu transisi yang mungkin. Hal ini membuat proses lexical analysis menjadi efisien dengan kompleksitas linear terhadap panjang input.

Selain itu, DFA memungkinkan implementasi lexer yang sederhana karena tidak memerlukan teknik backtracking yang kompleks. Setiap keputusan diambil berdasarkan state saat ini dan simbol input yang sedang dibaca, sehingga proses analisis dapat dilakukan secara sistematis dan konsisten.

# 2 Perancangan dan Implementasi

## 2.1 Perancangan DFA

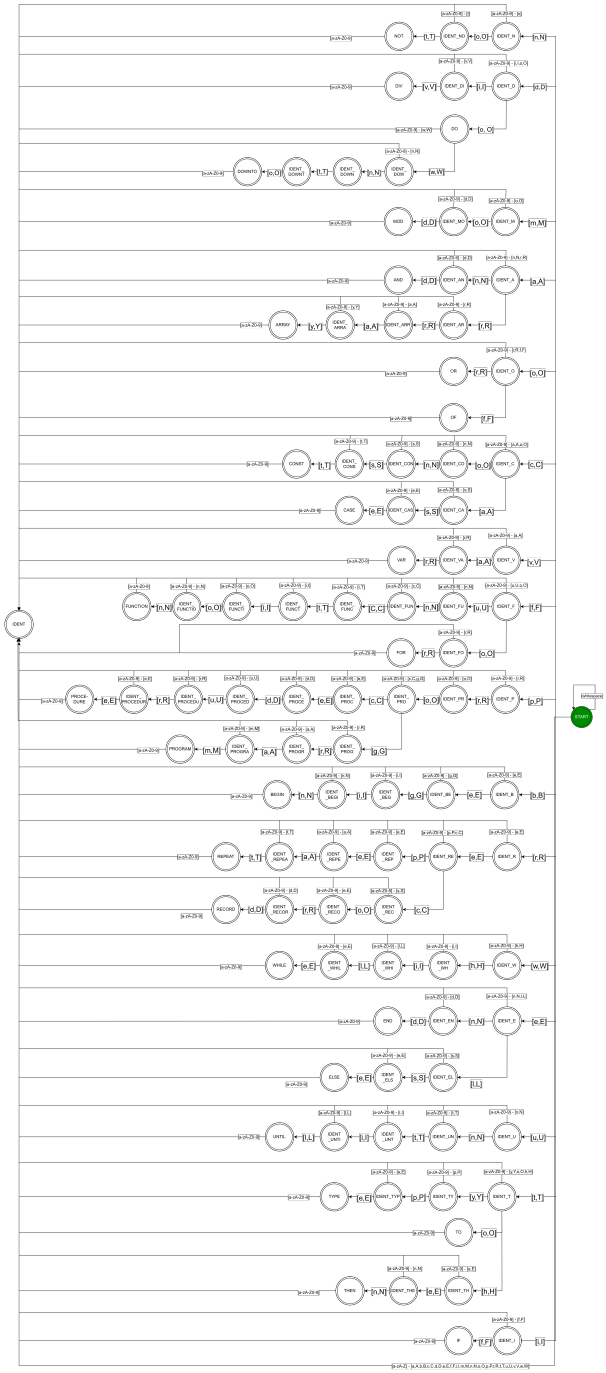

Figure 4: Diagram DFA bagian 1

*Selengkapnya lihat workspace pada draw.io (lihat Lampiran A.2).*

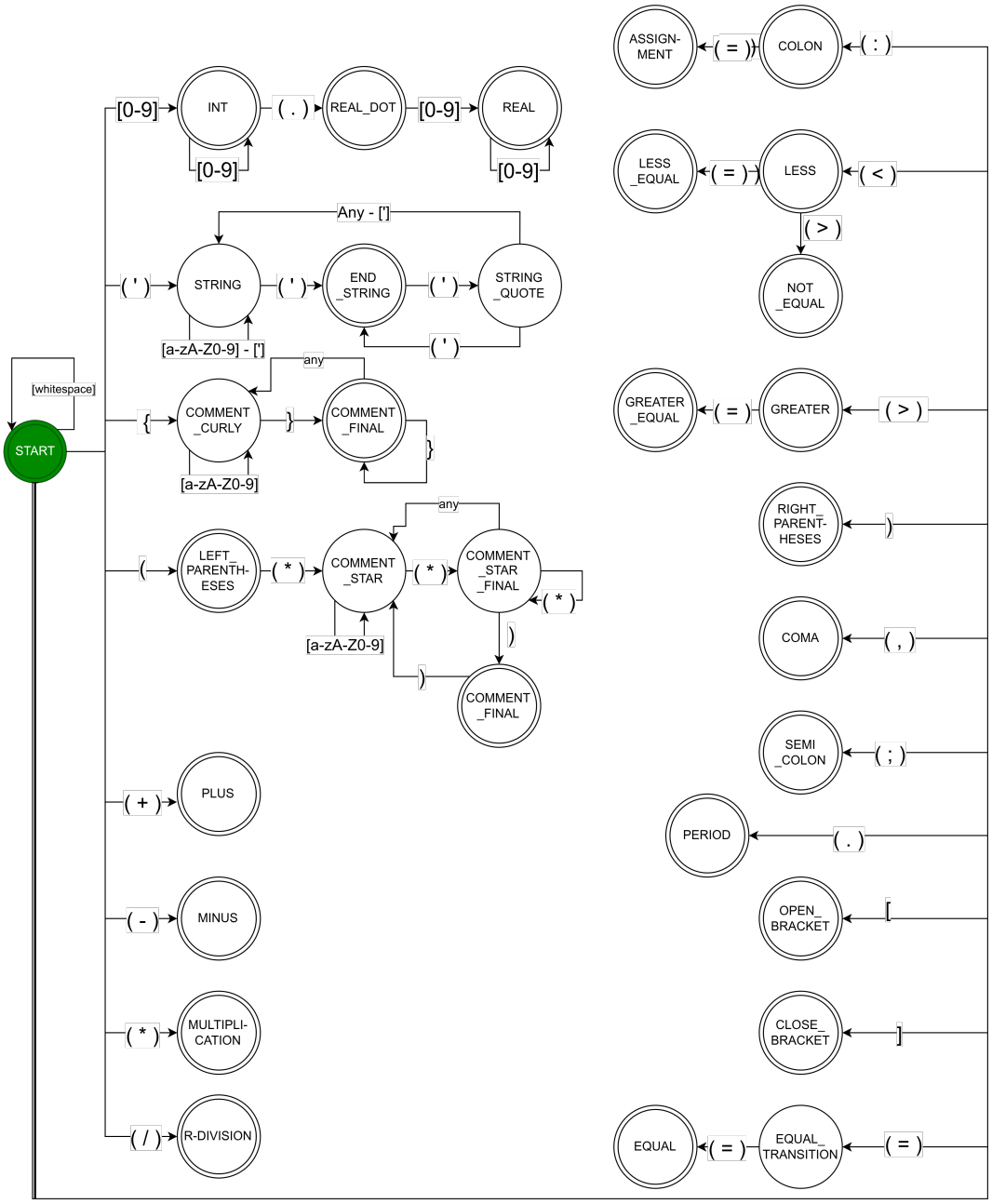


Figure 5: Diagram DFA bagian 2

*Selengkapnya lihat workspace pada draw.io (lihat Lampiran A.2).*

DFA yang dirancang digunakan untuk mengenali seluruh token dalam bahasa Arion berdasarkan spesifikasi. DFA bekerja dengan membaca karakter satu per satu dan berpindah state sesuai dengan jenis karakter yang diterima.

Secara umum, DFA dibagi menjadi beberapa kelompok utama:

1. **Identifier dan Keyword**
   - State awal menerima karakter huruf `[a-zA-Z]`
   - Transisi berulang pada `[a-zA-Z0-9]`

- Masuk ke state `IDENT` (final)
- Setelah token terbentuk, dilakukan pengecekan apakah termasuk keyword atau identifier

2. **Bilangan**
   - Integer:
     - State `INT` menerima digit `[0-9]`
     - Loop pada digit
   - Real:
     - Dari `INT` jika menemukan `.`, pindah ke `REAL_DOT`
     - Dilanjutkan digit menuju state `REAL`
3. **String**
   - Dimulai dari tanda petik tunggal `'`
   - Masuk ke state `STRING`
   - Menerima karakter `[a-zA-Z0-9]` dan whitespace
   - Ditutup dengan `'` menuju `END_STRING` (final)
4. **Comment**
   - Kurung kurawal:
     - { → `COMMENT_CURLY`
     - } → `COMMENT_FINAL`
   - Kurung `(* *)`:
     - `(` → cek `*`
     - `*` → `COMMENT_STAR`
     - `*)` → `COMMENT_STAR_FINAL` → `COMMENT_FINAL`
   - Komentar harus menggunakan pasangan yang sesuai, yaitu { } atau `(* *)`. Tidak diperbolehkan mencampur bentuk komentar, seperti { `*)` atau `(*` }.
5. **Operator dan Relasional**
   - Assignment:
     - `:` → `COLON`
     - `:=` → `ASSIGNMENT`
   - Relasional:
     - `<` → `LESS`
     - `<=` → `LESS_EQUAL`
     - `<>` → `NOT_EQUAL`
     - `>` → `GREATER`
     - `>=` → `GREATER_EQUAL`
     - `=` → `EQUAL`

  - `==` → `EQUAL` (dua karakter)

6. **Simbol dan Operator Aritmatika**
   - `+` → `PLUS`
   - `-` → `MINUS`
   - `*` → `MULTIPLICATION`
   - `/` → `R-DIVISION`
   - `(` → `LEFT_PARENTHESES`
   - `)` → `RIGHT_PARENTHESES`
   - `[` → `OPEN_BRACKET`
   - `]` → `CLOSE_BRACKET`
   - `,` → `COMMA`
   - `;` → `SEMI_COLON`

Whitespace diabaikan dan mengembalikan DFA ke state START.

## 2.2 Penjelasan Diagram DFA

Diagram DFA terdiri dari beberapa state utama:

- **START**: state awal
- **Intermediate states**: seperti INT, STRING, COMMENT
- **Final states**: seperti IDENT, REAL, END_STRING, COMMENT_FINAL

Setiap transisi ditentukan oleh kelas karakter (misalnya [a-zA-Z], [0-9], simbol khusus). DFA juga menerapkan konsep:

- **Maximal Munch**: mengambil token terpanjang
- **Lookahead**: untuk membedakan token seperti ‘:‘ dan ‘:=‘

## 2.3 Justifikasi Desain DFA

Perancangan DFA pada lexer ini tidak dilakukan secara acak, melainkan berdasarkan kebutuhan untuk membedakan pola token yang memiliki kemiripan bentuk namun berbeda makna. Berikut adalah justifikasi utama dari desain yang digunakan:

1. **Pemisahan state IDENT**

   Identifier dan keyword memiliki pola awal yang sama, yaitu diawali huruf dan dapat diikuti huruf atau angka. Oleh karena itu digunakan satu state `IDENT` untuk menangkap seluruh kemungkinan tersebut.

   Justifikasi:

- Menghindari duplikasi state untuk setiap keyword
- Lebih efisien: Beberapa keyword diproses sebagai `IDENT` terlebih dahulu
- Klasifikasi keyword dilakukan setelah token selesai (post-processing)

2. **Pemisahan INT, REAL_DOT, dan REAL**

   Bilangan real tidak dapat langsung dikenali dari awal karena diawali seperti integer. Oleh karena itu dibutuhkan state antara (`REAL_DOT`).

   Justifikasi:

   - Menghindari ambiguitas antara integer dan real
   - Memastikan format valid: digit harus ada setelah titik `.`
   - Sesuai prinsip DFA: setiap state merepresentasikan kondisi parsing yang spesifik

3. **State khusus STRING dan END_STRING**

   String memiliki delimiter khusus (`'`), sehingga membutuhkan state terpisah dari token lain.

   Justifikasi:

   - Isi string tidak mengikuti aturan token biasa
   - Perlu memastikan string ditutup dengan benar
   - Menghindari konflik dengan karakter lain seperti operator atau angka

4. **Pemisahan COMMENT_CURLY dan COMMENT_STAR**

   Terdapat dua format komentar: `{ }` dan `(* *)`. Keduanya memiliki pola pembukaan dan penutupan yang berbeda.

   Justifikasi:

   - DFA tidak dapat mengingat konteks kompleks, sehingga perlu state berbeda
   - Menghindari kesalahan penutupan komentar
   - Memastikan semua karakter dalam komentar diabaikan

5. **State antara untuk operator multi-karakter (COLON, LESS, GREATER)**

   Operator seperti `:=`, `<=`, `>=`, `<>` membutuhkan pembacaan lebih dari satu karakter.

   Justifikasi:

   - DFA membaca satu karakter, sehingga perlu state sementara (lookahead)
   - Menghindari salah klasifikasi (misal `:` vs `:=`)
   - Menjamin implementasi prinsip maximal munch (ambil token terpanjang)

6. **Pemisahan operator sederhana (PLUS, MINUS, dll)**

   Operator satu karakter langsung menuju state final tanpa state antara.

   Justifikasi:

   - Tidak membutuhkan lookahead

- Meminimalkan kompleksitas DFA

7. **Penggunaan START sebagai reset state**

   Semua token dimulai dari `START` setelah token sebelumnya selesai diproses.

   Justifikasi:

   - Memastikan independensi antar token
   - Mempermudah implementasi loop lexer

8. **Penerapan mekanisme retract (backtracking 1 karakter)**

   Digunakan ketika karakter terakhir yang dibaca bukan bagian dari token saat ini.

   Justifikasi:

   - Menjaga akurasi tokenisasi
   - Dibutuhkan karena DFA tidak menyimpan buffer input ke depan

## 2.4 Perancangan dan Implementasi Program

Program lexer diimplementasikan menggunakan bahasa C++ dan disusun secara modular untuk memisahkan tanggung jawab setiap komponen.

### 2.4.1 Modul DFA

```cpp
enum State
{
    STATE_START,

    STATE_IDENT,

    STATE_INT,       // [0-9]+
    STATE_REAL_DOT,  // [0-9]+'.'
    STATE_REAL,      // [0-9]+'.'[0-9]+

    STATE_STRING,
    STATE_END_STRING,

    STATE_COMMENT_CURLY,

    STATE_COMMENT_STAR,
    STATE_COMMENT_STAR_END,

    STATE_COMMENT_FINAL,

    STATE_COLON,       // ':'
    STATE_ASSIGNMENT,  // ':='

    STATE_LESS,              // '<'
    STATE_LESS_EQUAL,        // '<='
    STATE_NOT_EQUAL,         // '<>'
    STATE_GREATER,           // '>'
    STATE_GREATER_EQUAL,     // '>='
```

```
    STATE_EQUAL_TRANSITION, // '='
    STATE_EQUAL,            // '=='

    STATE_LEFT_PAREN,     // '('
    STATE_RIGHT_PAREN,    // ')'
    STATE_PLUS,           // '+'
    STATE_MINUS,          // '-'
    STATE_MULTIPLICATION, // '*'
    STATE_RDIV,           // '/'
    STATE_COMMA,          // ','
    STATE_SEMICOLON,      // ';'
    STATE_PERIOD,         // '.'
    STATE_OPENBRACK,      // '['
    STATE_CLOSEBRACK,     // ']'

    STATE_DEAD,
    STATE_FINAL,
    STATE_STRING_QUOTE
};

namespace DFA
{
    /**
     * Menentukan state berikutnya berdasarkan state saat ini dan
       karakter input.
     * Mengikuti aturan transisi DFA, dan mengembalikan STATE_DEAD jika
       tidak valid.
     */
    State getNextState(State current, char c);

    /**
     * Mengecek apakah suatu state adalah accepting state
     * (artinya lexeme bisa dijadikan token di state ini).
     */
    bool isAccepting(State state);

    /**
     * Mengonversi state DFA menjadi TokenType yang sesuai.
     */
    TokenType stateToTokenType(State state);
}
```

*Selengkapnya lihat file* ***dfa.hpp*** *dan* ***dfa.cpp*** *pada repository (lihat Lampiran A.1).*

Kedua file ini digunakan untuk mendefinisikan state-state dan transisi state sehingga merepresentasikan DFA. State-state didefinisikan dengan menggunakan enum sehingga setiap state didefinisikan dengan angka, misalnya STATE_INT bernilai 2. Transisi antar state menggunakan fungsi getNextState() yang berisi switch dengan parameter state. Selain getNextState(), terdapat juga fungsi isAccepting() dan stateToTokenType() yang masing-masing menerima parameter state. Fungsi isAccepting() digunakan mengetahui apabila state sekarang merupakan state yang valid atau tidak. Sedangkan, stateToTokenType() digunakan proses tokenisasi, dimana mengubah sebuah state menjadi sebuah token.

### 2.4.2 Modul Lexer

```cpp
#pragma once

#include "token.hpp"
#include "dfa.hpp"
#include <string>
#include <vector>

class Lexer
{
public:
    explicit Lexer(const std::string &source);

    std::vector<Token> scan();

private:
    std::string source;
    size_t pos;
    int line;
    int col;

    /**
     * Mengubah lexeme hasil DFA menjadi token dan menambahkannya ke list.
     *
     * - Identifier dicek apakah keyword atau bukan.
     * - String literal diproses (handle escape '' jadi ').
     * - Angka langsung dijadikan token sesuai tipe.
     * - Komentar diabaikan (tidak menghasilkan token).
     */
    void emitToken(std::vector<Token> &tokens, State state, const std::string &lexeme, int tokenLine, int tokenCol);
};
```

*Selengkapnya lihat file* `lexer.hpp` *dan* `lexer.cpp` *pada repository (lihat Lampiran A.1).*

Modul lexer ini yang mengandung file .hpp dan .cpp digunakan untuk melakukan analisis lexical. Untuk melakukan analisis lexical, modul lexer menggunakan fungsi-fungsi dan definisi dari modul dfa. Struktur modul lexel berupa sebuah class bernama lexer, dimana terdapat fungsi lexer() dan scan() dengan akses publik. Selain itu, terdapat atribut source, pos, line, dan col, lalu fungsi emitToken(). Atribut-atribut ini digunakan untuk membantu modul lexel menelusuri kode karakter demi karakter untuk melakukan tokenisasi. Fungsi scan() merupakan fungsi utama untuk melakukan tokenisasi, dimana fungsi ini akan loop terus hingga file program berakhir dan menangani kasus error. Fungsi emitToken() merupakan fungsi bantuan untuk fungsi scan(), dimana fungsi akan menangani kasus state yang terdiri lebih dari satu karakter, seperti integer, string, dll. Selain itu, fungsi ini akan mengubah state menjadi token dengan menggunakan fungsi stateToTokenType() dari modul lexel dan menambahkan token tersebut ke vektor token.

### 2.4.3 Modul Token

```cpp
#pragma once

#include <string>

enum TokenType
```

```
{
    // literal
    TOKEN_INTCON,  // 1.  konstanta integer
    TOKEN_REALCON, // 2.  konstanta bilangan real
    TOKEN_CHARCON, // 3.  konstanta karakter tunggal ('x')
    TOKEN_STRING,  // 4.  string ('abc')

    // operator logika
    TOKEN_NOTSY, // 5.  not
    TOKEN_ANDSY, // 12. and
    TOKEN_ORSY,  // 13. or

    // operator aritmatika
    TOKEN_PLUS,  // 6.  +
    TOKEN_MINUS, // 7.  -
    TOKEN_TIMES, // 8.  *
    TOKEN_IDIV,  // 9.  div
    TOKEN_RDIV,  // 10. /
    TOKEN_IMOD,  // 11. mod

    // operator relasional
    TOKEN_EQL, // 14. ==
    TOKEN_NEQ, // 15. <>
    TOKEN_GTR, // 16. >
    TOKEN_GEQ, // 17. >=
    TOKEN_LSS, // 18. <
    TOKEN_LEQ, // 19. <=

    // delimiter
    TOKEN_LPARENT,   // 20. (
    TOKEN_RPARENT,   // 21. )
    TOKEN_LBRACK,    // 22. [
    TOKEN_RBRACK,    // 23. ]
    TOKEN_COMMA,     // 24. ,
    TOKEN_SEMICOLON, // 25. ;
    TOKEN_PERIOD,    // 26. .
    TOKEN_COLON,     // 27. :
    TOKEN_BECOMES,   // 28. :=

    // keyword deklarasi
    TOKEN_CONSTSY,     // 29. const
    TOKEN_TYPESY,      // 30. type
    TOKEN_VARSY,       // 31. var
    TOKEN_FUNCTIONSY,  // 32. function
    TOKEN_PROCEDURESY, // 33. procedure
    TOKEN_ARRAYSY,     // 34. array
    TOKEN_RECORDSY,    // 35. record
    TOKEN_PROGRAMSY,   // 36. program

    // identifier
    TOKEN_IDENT, // 37. identifier

    // keyword kontrol
    TOKEN_BEGINSY,  // 38. begin
    TOKEN_IFSY,     // 39. if
    TOKEN_CASESY,   // 40. case
    TOKEN_REPEATSY, // 41. repeat
    TOKEN_WHILESY,  // 42. while
```

```
    TOKEN_FORSY,     // 43. for
    TOKEN_ENDSY,     // 44. end
    TOKEN_ELSESY,    // 45. else
    TOKEN_UNTILSY,   // 46. until
    TOKEN_OFSY,      // 47. of
    TOKEN_DOSY,      // 48. do
    TOKEN_TOSY,      // 49. to
    TOKEN_DOWNTOSY, // 50. downto
    TOKEN_THENSY,    // 51. then

    // komentar - dikenali tapi di-skip dari output
    TOKEN_COMMENT, // 52. { ... } atau (* ... *)

    // special
    TOKEN_ERROR, // token ga dikenali
    TOKEN_EOF    // end of file
};

struct Token
{
    TokenType type;
    std::string value;
    int line;
    int col;
};

inline std::string tokenTypeToString(TokenType type)
{
    switch (type)
    {
    case TOKEN_INTCON:
        return "intcon";
    case TOKEN_REALCON:
        return "realcon";
    case TOKEN_CHARCON:
        return "charcon";
    case TOKEN_STRING:
        return "string";
    case TOKEN_NOTSY:
        return "notsy";
    case TOKEN_PLUS:
        return "plus";
    case TOKEN_MINUS:
        return "minus";
    case TOKEN_TIMES:
        return "times";
    case TOKEN_IDIV:
        return "idiv";
    case TOKEN_RDIV:
        return "rdiv";
    case TOKEN_IMOD:
        return "imod";
    case TOKEN_ANDSY:
        return "andsy";
    case TOKEN_ORSY:
        return "orsy";
    case TOKEN_EQL:
        return "eql";
```

```
    case TOKEN_NEQ:
        return "neq";
    case TOKEN_GTR:
        return "gtr";
    case TOKEN_GEQ:
        return "geq";
    case TOKEN_LSS:
        return "lss";
    case TOKEN_LEQ:
        return "leq";
    case TOKEN_LPARENT:
        return "lparent";
    case TOKEN_RPARENT:
        return "rparent";
    case TOKEN_LBRACK:
        return "lbrack";
    case TOKEN_RBRACK:
        return "rbrack";
    case TOKEN_COMMA:
        return "comma";
    case TOKEN_SEMICOLON:
        return "semicolon";
    case TOKEN_PERIOD:
        return "period";
    case TOKEN_COLON:
        return "colon";
    case TOKEN_BECOMES:
        return "becomes";
    case TOKEN_CONSTSY:
        return "constsy";
    case TOKEN_TYPESY:
        return "typesy";
    case TOKEN_VARSY:
        return "varsy";
    case TOKEN_FUNCTIONSY:
        return "functionsy";
    case TOKEN_PROCEDURESY:
        return "proceduresy";
    case TOKEN_ARRAYSY:
        return "arraysy";
    case TOKEN_RECORDSY:
        return "recordsy";
    case TOKEN_PROGRAMSY:
        return "programsy";
    case TOKEN_IDENT:
        return "ident";
    case TOKEN_BEGINSY:
        return "beginsy";
    case TOKEN_IFSY:
        return "ifsy";
    case TOKEN_CASESY:
        return "casesy";
    case TOKEN_REPEATSY:
        return "repeatsy";
    case TOKEN_WHILESY:
        return "whilesy";
    case TOKEN_FORSY:
        return "forsy";
```

```
    case TOKEN_ENDSY:
        return "endsy";
    case TOKEN_ELSESY:
        return "elsesy";
    case TOKEN_UNTILSY:
        return "untilsy";
    case TOKEN_OFSY:
        return "ofsy";
    case TOKEN_DOSY:
        return "dosy";
    case TOKEN_TOSY:
        return "tosy";
    case TOKEN_DOWNTOSY:
        return "downtosy";
    case TOKEN_THENSY:
        return "thensy";
    case TOKEN_COMMENT:
        return "comment";
    case TOKEN_ERROR:
        return "ERROR";
    case TOKEN_EOF:
        return "EOF";
    default:
        return "UNKNOWN";
    }
}

inline bool tokenHasValue(TokenType type)
{
    return type == TOKEN_INTCON || type == TOKEN_REALCON ||
           type == TOKEN_CHARCON || type == TOKEN_STRING || type ==
               TOKEN_COMMENT ||
           type == TOKEN_IDENT || type == TOKEN_ERROR;
}

inline std::string tokenToString(const Token &tok)
{
    std::string result = tokenTypeToString(tok.type);
    if (tokenHasValue(tok.type))
    {
        result += "␣(" + tok.value + ")";
    }
    return result;
}
```

*Selengkapnya lihat file* ***token.hpp*** *pada repository (lihat Lampiran A.1).*

Modul ini hanya berisi pendefinisian setiap token dengan menggunakan enum dan beberapa fungsi bantuan. Fungsi tokenTypeToString() membantu untuk mencetak token dari token yang berupa enum dengan membuat mapping token dengan stringnya, misalnya token TOKEN_INTCON akan diubah menjadi string "intcon" untuk nanti dicetak sebagai output. Fungsi tokenHasValue() digunakan untuk mengetahui apakah token yang digunakan sekarang memiliki sebuah nilai, seperti integer, char, string, real, error, dan ident. Terakhir, fungsi tokenToString() menggunakan fungsi tokenTypeToString() untuk mencetak token.

### 2.4.4 Modul Utils

```
#pragma once

#include "token.hpp"
#include <string>

// konversi string ke lowercase
// dipakai buat keyword matching yang case-insensitive
std::string toLower(const std::string &str);

// cek apakah identifier ini keyword Arion atau bukan
// kalau iya -> return TokenType keyword-nya
// kalau bukan -> return TOKEN_IDENT
// case-insensitive: "BEGIN", "Begin", "begin" semua -> TOKEN_BEGINSY
TokenType lookupKeyword(const std::string &identifier);

// baca seluruh isi file ke string
// return true kalau berhasil, false kalau gagal
bool readFile(const std::string &path, std::string &content);

// tulis string ke file
// return true kalau berhasil, false kalau gagal
bool writeFile(const std::string &path, const std::string &content);
```

*Selengkapnya lihat file* ***utils.hpp*** *dan* ***utils.cpp*** *pada repository (lihat Lampiran A.1).*

Modul utils mengandung fungsi-fungsi yang digunakan untuk membantu memulaikan proses analisis lexel. Fungsi toLower() digunakan untuk mengubah semua huruf menjadi bentuk lower. Fungsi readFile() dan writeFile() digunakan masing-masing untuk membaca dan mengeluarkan teks dengan menggunakan path sehingga mengetahui file input dan lokasi untuk mengeluarkan output. Lalu, fungsi lookupKeyword() digunakan untuk mengubah string pada file program menjadi token, khususnya untuk state STATE_IDENT dengan terlebih dahulu menjalankan fungsi toLower().

### 2.4.5 Main Program

```
#include "lexer.hpp"
#include "utils.hpp"
#include <iostream>
#include <string>
#include <sstream>
#include <filesystem>
namespace fs = std::filesystem;

int main(int argc, char *argv[])
{
    if (argc < 2)
    {
        std::cerr << "Usage: " << argv[0] << " <input.txt> [output.txt]"
            << std::endl;
        return 1;
    }

    fs::path current = fs::canonical(argv[0]).parent_path();

```

```
    while (!fs::exists(current / "test"))
    {
        if (current == current.parent_path())
        {
            std::cerr << "Error: folder 'test' tidak ditemukan\n";
            return 1;
        }
        current = current.parent_path();
    }

    fs::path baseDir = current / "test";

    // input
    fs::path inputPath = argv[1];
    if (!inputPath.is_absolute())
    {
        inputPath = baseDir / inputPath;
    }

    // output
    fs::path outputPath;
    if (argc >= 3)
    {
        outputPath = argv[2];
        if (!outputPath.is_absolute())
        {
            outputPath = baseDir / outputPath;
        }
    }

    std::string source;
    if (!readFile(inputPath.string(), source))
    {
        std::cerr << "Error: tidak dapat membuka file '" << inputPath <<
            "'" << std::endl;
        return 1;
    }

    Lexer lexer(source);
    std::vector<Token> tokens = lexer.scan();

    std::stringstream output;
    for (const Token &tok : tokens)
    {
        if (tok.type == TOKEN_ERROR)
        {
            output << "ERROR (" << tok.value << ")" << std::endl;
        }
        else
        {
            output << tokenToString(tok) << std::endl;
        }
    }

    std::cout << output.str();

    if (!outputPath.empty())
    {
```

```
        if (!writeFile(outputPath.string(), output.str()))
        {
            std::cerr << "Error: tidak dapat menulis ke file '" << outputPath << "'" << std::endl;
            return 1;
        }
        std::cout << "\nOutput berhasil ditulis ke " << outputPath << std::endl;
    }

    return 0;
}
```

*Selengkapnya lihat file* ***main.cpp*** *pada repository (lihat Lampiran A.1).*

File main.cpp ini adalah file paling awal dan paling akhir dijalankan, dimana file ini mendefinisikan path input file dan output. Lalu, menjalankan analisis lexical dengan menjalankan fungsi scan(). Serta, mengeluarkan file output sesuai path output ditentukan.

### 2.4.6 Alur Eksekusi Program

Secara keseluruhan, alur kerja program adalah sebagai berikut:

1. Program dijalankan melalui `main.cpp`
2. Input dibaca karakter per karakter
3. Lexer memanggil DFA untuk menentukan state
4. Token dibentuk ketika mencapai state final
5. Hasil token ditampilkan atau disimpan

# 3 Pengujian

Berikut ini merupakan beberapa contoh pengujian kasus unik terhadap program.

## 3.1 Uji 1

Kasus unik: terdapat pendefinisian variabel dan operasi perkalian dalam satu program, sehingga menguji pengenalan keyword, identifier, dan operator aritmatika sekaligus.

```
program HitungLuasLingkaran;
const
  pi := 3.14;
var
  jari2: integer;
  luas: real;
begin
  jari2 := 10;
  luas := pi * jari2 * jari2;
end.
```

Figure 6: Input 1

```
programsy
ident (HitungLuasLingkaran)
semicolon
constsy
ident (pi)
becomes
realcon (3.14)
semicolon
varsy
ident (jari2)
colon
ident (integer)
semicolon
ident (luas)
colon
ident (real)
semicolon
beginsy
ident (jari2)
becomes
intcon (10)
semicolon
ident (luas)
becomes
ident (pi)
times
ident (jari2)
times
ident (jari2)
semicolon
endsy
period
```

Figure 7: Output 1

## 3.2 Uji 2

Kasus unik: terdapat penggunaan kondisi `if`, sehingga menguji pengenalan keyword dan operator relasional.

```
program CekKelulusan;
var
  nilaiTubes, nilaiUjian: integer;
  status: string;
begin
  nilaiTubes := 75;
  nilaiUjian := 80;

  if (nilaiTubes >= 70) and (nilaiUjian <> 0) then
    status := 'Lulus'
  else
    status := 'Mengulang';
end.
```

Figure 8: Input 2

```
programsy
ident (CekKelulusan)
semicolon
varsy
ident (nilaiTubes)
comma
ident (nilaiUjian)
colon
ident (integer)
semicolon
ident (status)
colon
ident (string)
semicolon
beginsy
ident (nilaiTubes)
becomes
intcon (75)
semicolon
ident (nilaiUjian)
becomes
intcon (80)
semicolon
ifsy
lparent
ident (nilaiTubes)
geq
intcon (70)
rparent
andsy
lparent
ident (nilaiUjian)
neq
intcon (0)
rparent
thensy
ident (status)
becomes
string ('Lulus')
elsesy
ident (status)
becomes
string ('Mengulang')
semicolon
endsy
period
```

Figure 9: Output 2

## 3.3 Uji 3

Kasus unik: terdapat perulangan `for` dan komentar, sehingga menguji keyword iterasi dan pengabaian komentar oleh lexer.

```
program TotalSum;
var
  i, sum: integer;
begin
  sum := 0;
  { Memulai perulangan dari 1 sampai 10 }
  for i := 1 to 10 do
  begin
    sum := sum + i; (* Tambahkan nilai i
                       ke dalam variabel sum *)
  end;
end.
```

Figure 10: Input 3

```
programsy
ident (TotalSum)
semicolon
varsy
ident (i)
comma
ident (sum)
colon
ident (integer)
semicolon
beginsy
ident (sum)
becomes
intcon (0)
semicolon
comment ( Memulai perulangan dari 1 sampai 10 )
forsy
ident (i)
becomes
intcon (1)
tosy
intcon (10)
dosy
beginsy
ident (sum)
becomes
ident (sum)
plus
ident (i)
semicolon
comment ( Tambahkan nilai i
                    ke dalam variabel sum )
endsy
semicolon
endsy
period
```

Figure 11: Output 3

## 3.4 Uji 4

Kasus unik: menggunakan `array` dan `type`, sehingga menguji deklarasi struktur data kompleks.

```
program Pendataan;
type
  Mahasiswa := record
    nilai: array [1..10] of real;
  end;
var
  mhs: Mahasiswa;
begin
  mhs.nilai[1] := 85.5;
end.
```

Figure 12: Input 4

```
programsy
ident (Pendataan)
semicolon
typesy
ident (Mahasiswa)
becomes
recordsy
ident (nilai)
colon
arraysy
lbrack
intcon (1)
period
period
intcon (10)
rbrack
ofsy
ident (real)
semicolon
endsy
semicolon
varsy
ident (mhs)
colon
ident (Mahasiswa)
semicolon
beginsy
ident (mhs)
period
ident (nilai)
lbrack
intcon (1)
rbrack
becomes
realcon (85.5)
semicolon
endsy
period
```

Figure 13: Output 4

## 3.5 Uji 5

Kasus unik: menggunakan operator `div` dan `not`, sehingga menguji operator aritmatika dan logika.

```
program LogikaDanFungsi;
var
  angka: integer;
  ganjil: boolean;
begin
  angka := 15;
  while (angka > 0) do
  begin
    ganjil := (angka mod 2 == 1) or not false;
    angka := angka div 2;
  end;
end.
```

Figure 14: Input 5

```
programsy
ident (LogikaDanFungsi)
semicolon
varsy
ident (angka)
colon
ident (integer)
semicolon
ident (ganjil)
colon
ident (boolean)
semicolon
beginsy
ident (angka)
becomes
intcon (15)
semicolon
whilesy
lparent
ident (angka)
gtr
intcon (0)
rparent
dosy
beginsy
ident (ganjil)
becomes
lparent
ident (angka)
imod
intcon (2)
eql
intcon (1)
rparent
orsy
notsy
ident (false)
semicolon
ident (angka)
becomes
ident (angka)
idiv
intcon (2)
semicolon
endsy
semicolon
endsy
period
```

Figure 15: Output 5

# 4 Kesimpulan dan Saran

## 4.1 Kesimpulan

Program lexical analyzer yang dikembangkan telah berhasil mengubah source code bahasa Arion menjadi token sesuai dengan spesifikasi yang ditentukan.

Implementasi menggunakan Deterministic Finite Automata (DFA) mampu mengenali berbagai jenis token seperti keyword, identifier, literal, operator, dan delimiter secara deterministik dan efisien. Pemisahan state pada DFA memungkinkan penanganan kasus khusus seperti bilangan real, string, komentar, serta operator multi-karakter dengan tepat.

Proses pembacaan karakter dilakukan secara berurutan dengan memanfaatkan mekanisme transisi state dan lookahead sederhana untuk membedakan token yang memiliki kemiripan bentuk.

Selain itu, struktur program yang modular mendukung kemudahan pengembangan untuk tahap selanjutnya.

Berdasarkan hasil pengujian, program telah mampu menangani berbagai kasus input sesuai spesifikasi, termasuk kombinasi keyword, operator, struktur kontrol, serta pengabaian komentar.

## 4.2 Saran

1. **Peningkatan error handling**
   Menambahkan deteksi error yang lebih spesifik seperti string tidak tertutup, komentar tidak lengkap, atau format angka tidak valid.

2. **Optimasi DFA**
   Menyederhanakan jumlah state tanpa mengurangi akurasi untuk meningkatkan efisiensi.

3. **Ekstensi fitur lexer**
   Menambahkan dukungan fitur tambahan seperti penanganan literal yang lebih kompleks jika dibutuhkan.

4. **Integrasi dengan modul berikutnya**
   Memastikan output token kompatibel dengan parser untuk mempermudah tahap syntax analysis.

5. **Pengujian lebih luas**
   Menambahkan pengujian edge case seperti input kosong, karakter tidak valid, dan input berukuran besar.

# A Lampiran

## A.1 Pranala Ke Repositori

https://github.com/mhps-null/MDC-Tubes-IF2224-2026

## A.2 Workspace Diagram Transisi DFA

https://drive.google.com/file/d/1MjCcgipbih4xMZfAtdd9oUyLAfq9YoDl/view?usp=sharing

## A.3 Pembagian Tugas

| NIM | Nama | Tugas |
|---|---|---|
| 13524041 | Nathan Adhika Santosa | Laporan, Source Code, Diagram |
| 13524053 | Muhammad Haris Putra Sulastianto | Laporan, Source Code, Diagram |
| 13524085 | Ariel Cornelius Sitorus | Source Code |
| 13524091 | Vara Azzara Ramli Pulukadang | Laporan |

Table 1: Pembagian Tugas

# Referensi

[1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. Compilers: Principles, techniques, and tools (2nd edition), 2006. Diakses: 2 April 2026. URL: https://edunexcontentprodhot.blob.core.windows.net/edunex/2025/81979-Formal-Language-Theory-and-Automata/362554-Module-01/file/1756694207024_Compilers-Principles-Techniques-and-Tools-2006.

[2] M.Sc. Dr. Judhi Santoso. Module 02: Formal language theory and automata, 2025. Diakses: 2 April 2026. URL: https://edunexcontentprodhot.blob.core.windows.net/edunex/2025/81979-Formal-Language-Theory-and-Automata/362555-Module-02/file/1757509229813_slide-02.

[3] GeeksforGeeks. Introduction of lexical analysis, n.d. Diakses: 30 Maret 2026. URL: https://www.geeksforgeeks.org/compiler-design/introduction-of-lexical-analysis/.

[4] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. Introduction to automata theory, languages, and computation (3rd edition), 2006. Diakses: 2 April 2026. URL: https://edunexcontentprodhot.blob.core.windows.net/edunex/2025/81979-Formal-Language-Theory-and-Automata/362554-Module-01/file/1756694207024_Introduction-to-Automata-Theory-Languages-and-Computation-Edition-3.

[5] TutorialsPoint. Deterministic finite automaton, n.d. Diakses: 30 Maret 2026. URL: https://www.tutorialspoint.com/automata_theory/deterministic_finite_automaton.htm.